Getting Started

CodeUI provides a fluent C# interface for constructing Unity UI Toolkit components.

---

1. Creating Components with BaseUI

Attach a `UIDocument` component to a GameObject and inherit from `BaseUI` to construct your interface inside `Generate`:

```
using CodeUI;
using CodeUI.Extensions;
using CodeUI.Tokens;
using UnityEngine;
using UnityEngine.UIElements;

public class SettingsScreen : BaseUI
{
    protected override void Generate(VisualElement root)
    {
        root.Column()
            .Padding(Spacing.MD)
            .WithChildren(
                new Label("Settings").FontSize(FontSize.LG),

                new Toggle("Mute Audio")
                    .OnValueChanged<Toggle, bool>(evt => AudioListener.pause
= evt.newValue),

                Spacer.Vertical(Spacing.MD),

                new Button(() => RebuildUI())
                    .SetText("Refresh")
            );
    }
}
```

---

2. Layout & Positioning

CodeUI provides shorthand extension methods for standard Flexbox properties:

Method	Description
`Row()`	Sets flex direction to horizontal (`FlexDirection.Row`)
`Column()`	Sets flex direction to vertical (`FlexDirection.Column`)
`Center()`	Centers elements horizontally and vertically (`Align.Center`, `Justify.Center`)
`Absolute()`	Sets position mode to absolute (`Position.Absolute`)
`Relative()`	Sets position mode to relative (`Position.Relative`)

---

3. Styling & Tokens

Standard design tokens ensure consistent spacing, typography, and layout boundaries across components:

- **Spacing**: `Spacing.XS` (4px), `Spacing.SM` (8px), `Spacing.MD` (16px), `Spacing.LG` (24px), `Spacing.XL` (32px), `Spacing.XXL` (48px)
- **FontSize**: `FontSize.XS` (12px), `FontSize.SM` (14px), `FontSize.MD` (16px), `FontSize.LG` (20px), `FontSize.XL` (28px), `FontSize.XXL` (40px)
- **Radius**: `Radius.None` (0px), `Radius.SM` (4px), `Radius.MD` (8px), `Radius.LG` (16px), `Radius.Full` (999px)

---

4. Event Handling

Events can be attached directly to visual elements during instantiation:

```
button.OnClick(() => Debug.Log("Button clicked"))
      .OnHover(isHovering => button.ToggleClasses(isHovering, "hovered"));
```

---

5. Animations & Transitions

Smooth CSS transitions can be defined using the type-safe `TransitionProperty` enum:

```
card.Transition(TransitionProperty.scale, durationSeconds: 0.2f, EasingMode.EaseOutCubic)
    .OnHover(hovered => card.Scale(hovered ? 1.05f : 1.0f));
```